

TRABAJO PARCIAL · HITO 1

NotaLang

Lenguaje de dominio específico para el
cálculo de notas y promedios ponderados

```
curso Compiladores {  
  PC1 : 15% = 14;  
  TP  : 20% = 16.5;  
  PC2 : 15%;  
  TF  : 25%;  
  EB  : 25%;  
}  
mostrar(necesito(Compiladores, 13));
```

CURSO	Teoría de Compiladores 2026-2
PROFESOR	José Luis Soncco Álvarez
INTEGRANTES	[Nombre 1] · [Nombre 2] · [Nombre 3] · [Nombre 4]

Problemática y motivación

Cada ciclo, los estudiantes calculan su promedio ponderado y **cuánto les falta para aprobar**, a mano o en una hoja de cálculo.

Pesos

que no suman
100 %

Notas

fuera del rango
0-20

Fórmulas

copiadas en la
celda equivocada

Una hoja de cálculo no conoce las reglas de una evaluación:
para ella, una nota de 25 es un número válido.

... hoja_de_calculo.xlsx

PC1	15%	14	
PC2	15%	25	← ¿nota 25?
EF	50%	12	
Total	80%		← ¿falta 20%?
Promedio	11.85		← sin aviso

Objetivos

Objetivo general: implementar en ANTLR4 los analizadores léxico, sintáctico y semántico de NotaLang.

01

Léxico

Definir tokens y lexemas e implementarlos con expresiones regulares

02

Sintáctico

Diseñar gramáticas libres de contexto y validarlas con derivaciones más a la izquierda

03

Semántico

Implementar la tabla de símbolos y verificar tipos y reglas del dominio

04

Pruebas

Probar cada analizador con entradas válidas e inválidas

Propuesta: NotaLang

```
curso Calculo {  
    PC : 40% = 9;  
    EF : 60%;  
}  
  
var meta : decimal = 10.5;  
mostrar("Promedio: ", promedio(Calculo));  
mostrar("Necesito: ", necesito(Calculo, meta))  
  
si (promedio(Calculo) >= meta) {  
    mostrar("Aprobado");  
}
```

- Cursos y evaluaciones **declarados directamente**
- Funciones: `promedio` y `necesito`
- Acceso a notas: `Calculo.EF`
- Tipos: `entero` `decimal` `texto` `logico` `curso`
- El compilador **valida las reglas del dominio** antes de ejecutar

Construcciones del lenguaje

C1 · Declaración de variables

```
var nota : decimal = 14.5;
```

C2 · Asignación

```
Compiladores.PC1 = 15;
```

C3 · Expresiones

```
nota >= 10.5 y no fin
```

C4 · Selectiva

```
si (p >= 10.5) { ... } sino { ... }
```

C5 · Iterativas

```
mientras (i < 10) { ... }  
para i desde 1 hasta n { ... }
```

C6 · Entrada / salida

```
leer(x);  
mostrar("Prom: ", promedio(C));
```

C7 · Declaración de curso — propia del dominio

```
curso Calculo { PC : 40% = 12; EF : 60%; }
```

Analizador léxico

Categoría	Lexemas
Palabras clave	var curso si sino mientras para desde hasta leer mostrar promedio necesito
Tipos	entero decimal texto logico
Lógicos	verdadero falso y o no
Operadores	+ - * / mod < > <= >= == != =
Delimitadores	() { } : ; , .

Cada lexema fijo es un token: **VAR:** var · **SI:** si · **MAYOR_IG:** >= · **PUNTO_COMA:** ;

Token	Expresión regular
NUM_ENT	[0-9]+
NUM_DEC	[0-9]+ '.' [0-9]+
PORCENTAJE	[0-9]+ ('.' [0-9]+)? '%'
CADENA	'"' ~["\r\n]* "'
ID	[a-zA-Z_] [a-zA-Z_0-9]*
Ignorados	espacios y comentarios (// , /* */)

Carácter inválido → **ERROR_LEXICO** con línea y columna.

Gramática

programa, declaraciones y entrada/salida

PROGRAMA

programa → *sentencias* **EOF**
sentencias → *sentencia* *sentencias* | ϵ
sentencia → *declVar* | *asignacion* | *siStmt* |
mientrasStmt | *paraStmt* | *leerStmt* |
mostrarStmt | *declCurso* | *bloque*
bloque → { *sentencias* }

C1 · DECLARACIÓN

declVar → **var** *id* : *tipo inicializacion* ;
tipo → **entero** | **decimal** | **texto** | **logico**
inicializacion → = *expr* | ϵ

C2 · ASIGNACIÓN

asignacion → *destino* = *expr* ;
destino → *id* | *id* . *id*

C6 · ENTRADA Y SALIDA

leerStmt → **leer** (*destino*) ;
mostrarStmt → **mostrar** (*argumentos*) ;
argumentos → *expr* *masArgumentos*
masArgumentos → , *expr* *masArgumentos* | ϵ

C7 · CURSO

declCurso → **curso** *id* { *evaluaciones* }
evaluaciones → *evaluacion* *evaluaciones* | *evaluacion*
evaluacion → **id** : **porcentaje** *notaOpcional* ;
notaOpcional → = *expr* | ϵ

Gramática expresiones y control

C3 · EXPRESIONES

<i>expr</i>	→ <i>expr</i> o <i>exprY</i> <i>exprY</i>
<i>exprY</i>	→ <i>exprY</i> y <i>exprNo</i> <i>exprNo</i>
<i>exprNo</i>	→ no <i>exprNo</i> <i>exprRel</i>
<i>exprRel</i>	→ <i>exprArit</i> <i>opRel</i> <i>exprArit</i> <i>exprArit</i>
<i>exprArit</i>	→ <i>exprArit</i> <i>opSuma</i> <i>termino</i> <i>termino</i>
<i>termino</i>	→ <i>termino</i> <i>opMul</i> <i>factor</i> <i>factor</i>
<i>factor</i>	→ (<i>expr</i>) - <i>factor</i> num_ent num_dec cadena verdadero falso id id . id promedio (<i>id</i>) necesito (<i>id</i> , <i>expr</i>)
<i>opRel</i>	→ < > <= >= == !=
<i>opSuma</i>	→ + -
<i>opMul</i>	→ * / mod

C4 · SELECTIVA

<i>siStmt</i>	→ si (<i>expr</i>) <i>bloque</i> <i>sinoParte</i>
<i>sinoParte</i>	→ sino <i>bloque</i> sino <i>siStmt</i> ϵ

C5 · ITERATIVAS

<i>mientrasStmt</i>	→ mientras (<i>expr</i>) <i>bloque</i>
<i>paraStmt</i>	→ para <i>id</i> desde <i>expr</i> hasta <i>expr</i> <i>bloque</i>

Un no-terminal por nivel de precedencia · llaves obligatorias: sin ambigüedad del *sino* colgante

Derivación más a la izquierda

Entrada: `curso Calculo { PC : 40% = 12; EF : 60%; }`

declCurso

$\Rightarrow_{lm}$ `curso id { evaluaciones }`
 $\Rightarrow_{lm}$ `curso id { evaluacion evaluaciones }`
 $\Rightarrow_{lm}$ `curso id { id : porcentaje notaOpcional ; evaluaciones }`
 $\Rightarrow_{lm}$ `curso id { id : porcentaje = expr ; evaluaciones }`
 $\Rightarrow_{lm}^*$ `curso id { id : porcentaje = num_ent ; evaluaciones }`
 $\Rightarrow_{lm}$ `curso id { id : porcentaje = num_ent ; evaluacion }`
 $\Rightarrow_{lm}$ `curso id { id : porcentaje = num_ent ; id : porcentaje notaOpcional ; }`
 $\Rightarrow_{lm}$ `curso id { id : porcentaje = num_ent ; id : porcentaje ; }`

$\Rightarrow_{lm}^*$ *expr* $\Rightarrow^*$ `num_ent`, derivada en la gramática de expresiones (C3)

9

gramáticas (C5 y C6 en dos)

36

derivaciones, 4 por gramática

45

ejemplos reconocidos sin errores

Analizador semántico

E1

No declarado

`b = a + 1;`

E2

Ya declarado

`var x : entero; x2`

E3

Tipos incompatibles

`var e : entero = 3.5;`

E4

Condición no lógica

`si (n) { ... }`

E5

Pesos no suman 100 %

`PC1 : 30%; EF : 50%;`

E6

Nota fuera de 0-20

`T1 : 50% = 25;`

E7

Evaluación inexistente

`Ingles.ESCRITO = 15;`

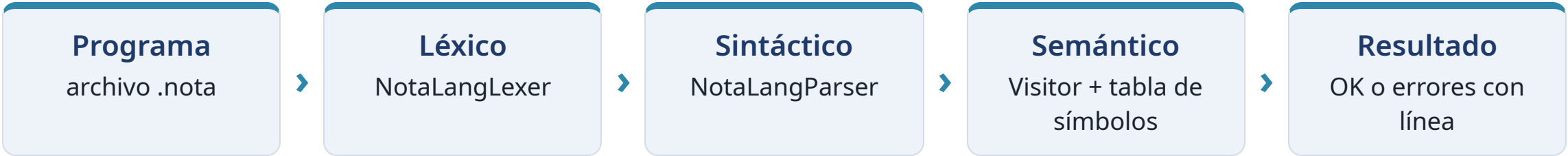
E8

Operación inválida

`"a" * 2`

- Errores del dominio · Visitor de ANTLR + tabla de símbolos con ámbitos anidados (bloques y `para`)

Implementación y pruebas



Analizador	Entradas	Resultado
Léxico	ok1-ok3 · err_lex1-2	Detecta # (1:19) y la comilla sin cerrar (1:21)
Sintáctico	ok1-ok3 · err_sin1-3	; y) faltantes, curso sin evaluaciones
Semántico	ok1-ok3 · err_sem1-8	Cada err_semN reporta el error EN con su línea

```
make genera el lexer y el parser desde NotaLang.g4 · make test ejecuta las 16 entradas de tests/
```

Demostración

```
$ python3 main.py tests/ok3_mientras.nota --tabla
Análisis léxico y sintáctico: OK
```

```
=== TABLA DE SÍMBOLOS ===
global  Calculo  curso  línea 1
        evaluaciones={'PC': 40.0, 'EF': 60.0}
global  ef       entero  línea 6
global  ok       logico  línea 7
Análisis semántico: OK
```

```
$ python3 main.py tests/err_sem5_pesos.nota
Análisis léxico y sintáctico: OK
```

```
=== ERRORES SEMÁNTICOS ===
[Error Semántico E5, línea 1]
Los pesos del curso 'Quimica' suman 80%
(deben sumar 100%)
```

```
$ python3 main.py tests/err_lex1.nota
=== ERRORES LÉXICOS ===
[Error Léxico, línea 1:19] carácter no reconocido '#'
$ python3 main.py tests/err_sin1.nota
=== ERRORES SINTÁCTICOS ===
[Error Sintáctico, línea 2:0] missing ';' at 'mostrar'
```

Conclusiones

Fase de análisis completa

Analizadores léxico, sintáctico y semántico implementados en ANTLR4 y probados

Errores del dominio

La nota 25 y el 80 % de la hoja de cálculo ahora son errores de compilación (E6 y E5)

Gramáticas validadas

36 derivaciones más a la izquierda sobre 9 gramáticas

Próximos pasos: generación de código para ejecutar `promedio` y `necesito`.

Gracias

¿Preguntas?